Exact Posteriors for Values, Exact Costs for Choices: Two-Stage Learned Branching for Binary Linear Systems

Gwang-Jong Ko[a,*], Taesu Cheong[a,†], In-Chan Choi[a,†]

[a]School of Industrial and Management Engineering, Korea University, Seoul, Republic of Korea

Abstract

A binary linear system (BLS) asks for $\mathbf{x} \in \{0,1\}^n$ with $A\mathbf{x} = \mathbf{b}$, $A \in \{0,1\}^{m\times n}$; on market-split-style instances the linear relaxation is uninformative and a complete search spends its effort on branching decisions. We present a branching heuristic learned in two stages from two exact signals, each answering a different question. Stage 1 answers *which value* a variable should take: on instances small enough to enumerate every solution, the exact conditional marginal $\Pr[x_j{=}1 \mid A, \mathbf{b}, \text{partial assignment}]$ is computable, and because conditioning a BLS on a partial assignment is identical to reducing it, one network trained on reduced instances predicts at every node of the tree. The marginals also partition a node's variables into those on which a wrong branch empties the subtree and those on which it does not, and only the former can cost anything. Stage 2 answers *which variable* to branch on: the cost of a branching decision — the number of nodes its subtree expands — is produced exactly by the search itself, and an actor-critic fine-tuning on that cost, initialized at the Stage-1 rule with the marginal network frozen, changes only the branching order. On a 100-instance 21×60 set with matched wall-clock budgets and three seeds, Stage 1 is 2.0–2.3× faster than LP-guided branching with 5.5× fewer nodes; Stage 2 is a further 2.0–2.4× faster with 2.3–2.4× fewer nodes ($p < 0.001$), replicates across training seeds, and transfers in size in both directions: a policy fine-tuned at 18×50 is 2.0× better than the supervised rule at 21×60, and both fine-tuned policies solve 30/30 instances at 24×70 2.3–3.3× faster than the supervised rule, which the LP rule solves 23–25/30; at 28×80 they double the solve rate within budget (18/30 against 8/30), where the useful window closes. The fine-tuned policy branches on a different variable than the confidence rule at 95% of held-out states, typically one the network is *less* sure about; training Stage 1 on deployment-sized states instead, where its prediction margin lives, doubles the tree. The probability that a branch is right and the cost of it being wrong are different objectives, and the search minimizes the second. We report what the method does not do: CP-SAT remains 9–10× faster at 21×60, and exact supervision is available only where enumeration completes.

Keywords: Binary linear systems, Learning to branch, Exact conditional marginals, Reinforcement learning, Tree search, Graph neural networks

1. Introduction

A binary linear system (BLS) is the feasibility problem

$$\text{find } \mathbf{x} \in \{0,1\}^n \text{ such that } A\mathbf{x} = \mathbf{b}, \qquad A \in \{0,1\}^{m\times n},\ \mathbf{b} \in \mathbb{Z}^m, \tag{1}$$

whose solution set we write $\mathcal{S}$. It arises in laser-based non-destructive testing, where a binary occupancy image is reconstructed from integer projections, and in its market-split form [5] it is a standard hard family: the relaxation polytope is large and its vertices are far from $\{0,1\}^n$, so bounding prunes little and a complete search lives or dies by its branching decisions. Lattice reduction [1] solves many such instances outright and constraint solvers with clause learning solve the rest, but neither makes the branching decision itself less expensive to get right.

We ask what signal a learned branching heuristic for (1) should be trained on, and we find that the answer has two parts because a branching decision has two parts. *Which value* a chosen variable should try first is a question about where the solutions are, and it has an exact answer on instances small enough to enumerate: the conditional marginal of x_j given the assignments made so far. *Which variable* to branch on is a question about the cost of the subtree that follows, and it has an exact answer too, produced by the search itself as it backtracks. The first signal is a probability; the second is a count. Our method trains on both, in that order, and the paper's central empirical finding is that they are not interchangeable: improving the probability where it matters most does not shrink the tree, while optimizing the cost does, and the cost-optimal policy branches on variables the probability model is *less* sure about.

Contributions..

1. **Exact conditional marginals as a supervision signal at every node of a complete search.** Training a network on exact marginals of an enumerated solution set is not new (NSNet does it for SAT, 14); what is new is where the target is used. Conditioning (1) on a partial assignment is identical to reducing it (Proposition 1), so the same marginals are available at every interior node, and they

*Corresponding author.

†Footnote text for the † mark to be supplied by the authors.

partition a node's variables into those that can cost a backtrack (FORCED) and those that cannot (OPEN). The FORCED share rises from 6% at the root to 88% at depth 8; a network trained on root states is worse than relaxation rounding inside the tree, one trained in the tree is better. The network only guides; it never fixes a variable.

2. **An exact cost signal for choices, read off the search.** Subtree size as the per-decision return of a branching policy is established [6, 23]; our contribution is the starting point and the scope. An actor-critic fine-tuning of the branching order on per-decision subtree costs, initialized at the posterior-derived rule with the marginal network frozen so that only the order can change, expands 2.3–2.4× fewer nodes and is 2.0–2.4× faster than the supervised rule on 100 instances ($p < 0.001$, two training seeds), and transfers to sizes below and above the one it was trained at, up to 24×70 where it is 2.3–3.3× faster and where the supervised rule alone no longer beats the LP rule in time.
3. **Evidence that the two signals capture different things.** A residual measurement shows the confidence rule picks the cheapest of its five candidates only 22% of the time; training the marginal network on deployment-sized states raises its accuracy where its margin lives by one point and doubles the tree; the fine-tuned policy disagrees with the confidence rule at 95% of states.
4. **A controlled end-to-end evaluation**, with solution multiplicity matched across sizes, matched wall-clock budgets, warm-started relaxations on every arm, three seeds, a 100-instance test set, comparisons against CP-SAT and SCIP, and a deployed cascade with lattice reduction in front.

2. Related work

Marginals of the solution set as a branching signal.. The idea that the fraction of solutions agreeing with a decision is the quantity to branch on predates learning. Counting-based search [21] branches on the variable–value pair of maximum *solution density*, computed exactly inside individual constraints; belief-propagation-guided decimation [17] fixes the most biased variable under approximate marginals recomputed after every fixing. Learned versions replace the counting by a network. NSNet [14] trains a BP-structured graph network on exact marginals of the satisfying assignments of a SAT formula, obtained by enumerating them, and rounds the estimates into an initial assignment for local search; MIP-GNN [11], Neural Diving [18] and predict-and-search [9] predict per-variable biases from collected near-optimal solutions and use them to fix variables, warm-start or restrict a search. Our Stage 1 takes the same target as NSNet — exact marginals over the full solution set — and differs in where and how it is used: the marginals are conditional, carried to every node of a complete search by the reduction identity; they guide branching without fixing anything; and the FORCED/OPEN partition, the ceiling analysis and the controlled in-tree-versus-root comparison have, to our knowledge, no counterpart in that work.

Learning to branch.. Khalil et al. [12] and Gasse et al. [7] imitate strong branching, the latter with a bipartite graph network; the gain is cheap access to an expert's ranking, and imitation targets that ranking rather than the tree size it is meant to reduce. Optimizing tree size directly is the subject of a line of reinforcement-learning work: FMSTS [6] uses the size of a node's subtree as an observable Q-value, shows that under depth-first search minimizing every subtree minimizes the whole tree, and learns from scratch; tree MDPs [23] formalize the per-subtree credit assignment and derive a tree policy gradient; retro branching [20] learns from retrospectively extracted subtree trajectories; SORREL [28] and PPO-based hybrids initialize from demonstrations. Neuro# [26] trains a graph network on the residual formula of a #SAT solver by evolution strategies to minimize the number of branching decisions, and generalizes to larger instances. Our Stage 2 uses the FMSTS/tree-MDP credit assignment with a plain policy gradient [32]; what it adds is the starting point and the scope — a rule derived from exact marginals, frozen, with only the branching order learned — which is what lets the improvement be attributed to the order, and the finding that the learned order moves away from the most confident variables. The classical statements of that objective are the fail-first principle [10] and impact-based search [22], which prefers the variable whose assignment shrinks the search space most. In constraint programming, value ordering has been learned from solutions [25] and by deep Q-learning with a graph network [24]; we instead take the value from the exact posterior and learn only the variable. Direct satisfiability prediction has a weaker record [30, 15], and message-passing networks provably cannot separate some feasible/infeasible pairs [4]; we never ask the network for a verdict.

Complete solvers and lattice methods.. Branch-and-bound gains little on market-split instances because the construction removes what bounding relies on [5]. Constraint solvers with clause learning [16, 19] manufacture information as they descend, which is why we study the interior of the tree. The Aardal–Hurkens–Lenstra embedding [1, 2] turns LLL/BKZ reduction [13, 29] into a certificate-producing first stage, and lattice enumeration remains the state of the art on the canonical benchmark: Wassermann [27] solves QOBLIB market-split instances up to $m = 14$ on one CPU where branch-and-cut stops at $m = 7$. Our instances are a different regime (solution multiplicity matched across sizes, $m/n \approx 0.35$), and our contribution is a branching methodology rather than a record on that benchmark; lattice reduction runs first in our pipeline.

Amortized inference.. A learned map that replaces a per-instance computation by a single forward pass is amortized optimization [3, 8]. Both stages are of this kind: the marginal network amortizes LP-probing (one relaxation per variable) and the cost head amortizes the subtree costs that only a completed search reveals.

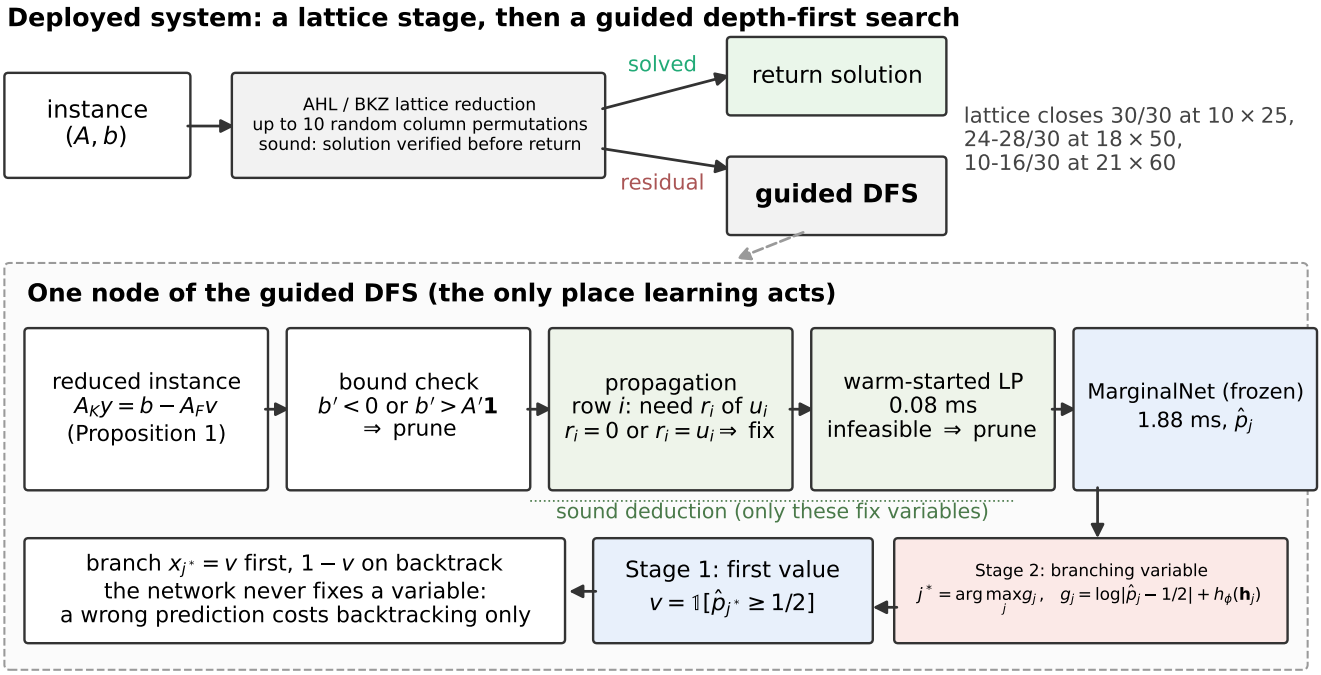


Figure 1: The deployed system. A lattice stage runs first and returns a verified solution when it succeeds; the guided depth-first search sees the residual. Inside a search node, sound deduction (bound check, propagation, the relaxation's infeasibility test) is what may fix variables or prune, and the learned components only choose the branching variable and the value tried first. The frozen marginal network supplies $\hat{p}_j$; Stage 1 takes the value from it, Stage 2 takes the variable from the cost-tuned policy head. Neither fixes a variable, so a wrong prediction costs backtracking and nothing else.

3. Method

3.1. Setting and search

Given (1), the deployed solver has two stages (Figure 1). The first is lattice reduction in the Aardal–Hurkens–Lenstra embedding [1], run under up to ten random column permutations; when a reduced basis yields a short vector of the right form, the corresponding 0/1 vector is checked against $A\mathbf{x} = \mathbf{b}$ and returned, so the stage is sound and produces a certificate. It closes 30/30 instances at 10×25, 24–28/30 at 18×50 and 10–16/30 at 21×60 (Section 4.5); its cost is 0.09–0.16 s per instance, which is 1% of end-to-end time at 21×60. Everything it does not close goes to the second stage.

The second stage is a depth-first search that maintains a partial assignment $\mathbf{x}_F = \mathbf{v}$ on a set F of fixed variables and a set K of free ones. At each node it (i) forms the reduced instance $A_K\mathbf{y} = \mathbf{b} - A_F\mathbf{v}$ (Proposition 1) and prunes if any $b'_i < 0$ or $b'_i > \sum_j a'_{ij}$; (ii) runs unit propagation, which on a 0/1 row reads "exactly r_i of the u_i still-unassigned variables in this row must be 1" and fixes them all when $r_i = 0$ or $r_i = u_i$, pruning on contradiction; (iii) solves the node relaxation, warm-started from the parent basis, and prunes if it is infeasible — a sound test strictly stronger than propagation, since it sees linear combinations of rows rather than one row at a time; and (iv) branches on $j \in K$ with a first value v, exploring $x_j = v$ first and $x_j = 1 - v$ on backtracking. Steps (i)–(iii) are the only ones that *fix* variables or prune; the learned components enter only at step (iv). A wrong prediction therefore costs backtracking and nothing else, and the search remains complete. LP-probing — fixing x_j to a value and pruning the opposite one when the relaxation becomes infeasible — is the sound procedure that detects the same variables the network predicts, at one relaxation per variable; Section 3.5 compares their costs.

3.2. Conditioning is reduction; which decisions cost anything

Let $\mathcal{S}(F, \mathbf{v}) = \{\mathbf{x} \in \mathcal{S} : \mathbf{x}_F = \mathbf{v}\}$ be the solutions consistent with the node and $p_j = \Pr[x_j{=}1 \mid A, \mathbf{b}, \mathbf{x}_F{=}\mathbf{v}] = |\{\mathbf{x} \in \mathcal{S}(F, \mathbf{v}) : x_j{=}1\}|/|\mathcal{S}(F, \mathbf{v})|$ the conditional marginal, the posterior under the uniform prior on $\mathcal{S}$ that a planted-solution generator induces.

Proposition 1. *With A_F, A_K the column submatrices indexed by F, K, the map $\mathbf{x} \mapsto \mathbf{x}_K$ is a bijection from $\mathcal{S}(F, \mathbf{v})$ onto $\{\mathbf{y} \in \{0,1\}^{|K|} : A_K\mathbf{y} = \mathbf{b} - A_F\mathbf{v}\}$. Hence the conditional marginals of the original instance are the ordinary marginals of the reduced instance.*

Proof. $A\mathbf{x} = A_F\mathbf{x}_F + A_K\mathbf{x}_K$; with $\mathbf{x}_F = \mathbf{v}$ fixed, $A\mathbf{x} = \mathbf{b}$ iff $A_K\mathbf{x}_K = \mathbf{b} - A_F\mathbf{v}$, and $\mathbf{x}_F$ is determined, so the map is a bijection; the marginal statement follows by counting. □

Two consequences (Figure 2). First, one architecture, one label generator and one training loop serve every node: a depth-d training state is stored as an instance on $n - d$ variables. Second, $p_j \in \{0, 1\}$ means every surviving solution agrees on x_j; branching the other way empties the subtree. We call such variables FORCED and the rest OPEN: either branch of a OPEN variable still

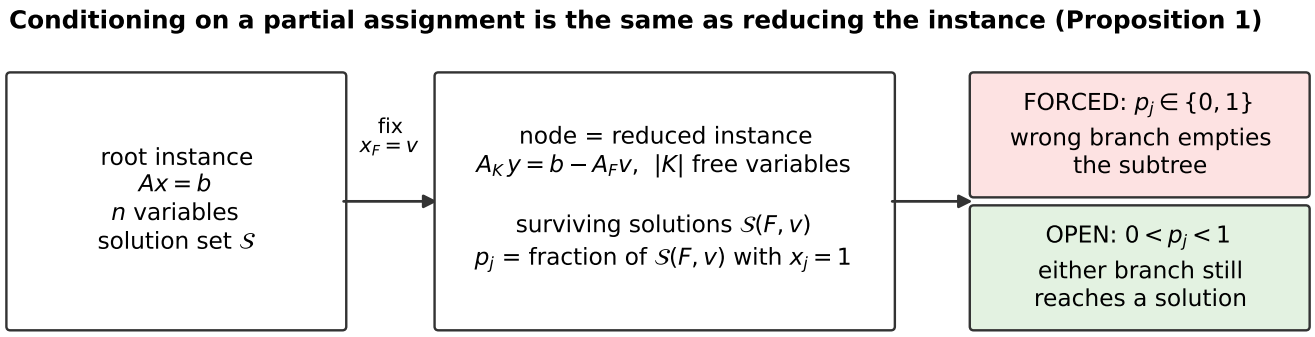


Figure 2: Proposition 1 and the partition it induces. A node with $\mathbf{x}_F = \mathbf{v}$ is the smaller BLS $A_K\mathbf{y} = \mathbf{b} - A_F\mathbf{v}$; its conditional marginals are the ordinary marginals of that instance. Variables on which every surviving solution agrees are FORCED; only branching against them costs a backtrack.

reaches a solution, so a wrong first value there costs nothing beyond exploring a subtree that contains one. Only FORCED errors cost a backtrack. Their share grows as the search descends and $|\mathcal{S}(F, \mathbf{v})|$ shrinks — 6.3% at the root of a 10×25 instance, 87.9% at depth 8 (Section 4.2) — which is why prediction is trained and evaluated inside the tree.

3.3. Stage 1: exact conditional marginals as targets

On instances small enough to enumerate (10×25 here), $\mathcal{S}$ is obtained with the all-solutions mode of a constraint solver and p_j computed by counting. Only enumerations that run to exhaustion (`OPTIMAL` or `INFEASIBLE`) are accepted; a time-limited enumeration with solutions already collected would substitute a truncated $\mathcal{S}$ and bias every label. Training states are reachable in-tree states: a depth $d \in \{0, \dots, 12\}$ is drawn uniformly, a prefix of an LP-confidence ordering is fixed to the values of a randomly chosen actual solution, and the reduced instance is enumerated in turn to give p_j on its free variables. The network f_θ (MarginalNet) is a bipartite variable–constraint graph network with graph attention [31]: three features per variable node (its LP value, column density, fractionality), three per constraint node (right-hand side, row density, LP residual, each normalized by row size), hidden width 64, four rounds of attention in each direction with residual connections and layer normalization, and one logit per variable; 21,761 parameters. It is trained by cross-entropy against the soft targets p_j,

$$\mathcal{L}_1 = -\tfrac{1}{|K|} \sum_{j \in K} [p_j \log \hat{p}_j + (1 - p_j) \log(1 - \hat{p}_j)], \qquad (2)$$

minimized exactly at $\hat{p} = p$. Replacing p_j by the planted solution, the usual choice, is a noisy surrogate whenever $|\mathcal{S}| > 1$. At a node the network is evaluated once; the supervised branching rule is

$$j^\star = \arg\max_{j \in K} |\hat{p}_j - \tfrac{1}{2}|, \qquad v = \mathbb{1}[\hat{p}_{j^\star} \geq \tfrac{1}{2}]. \qquad (3)$$

3.4. Stage 2: exact subtree costs as the objective

Rule (3) maximizes the chance that the first child contains a solution. It says nothing about how much the search pays when it does not, nor how much smaller the children are; both are properties of the subtree, and a depth-first search produces them exactly. Let $c(u)$ be the number of nodes expanded below node u until a solution is found there or the subtree is exhausted; the search returns it when it backtracks out of u. Stage 2 fine-tunes the branching order on $-\log c(u)$.

Decision process.. A state is the reduced instance after propagation; an action is a free variable j; the first value stays $\mathbb{1}[\hat{p}_j \geq \frac{1}{2}]$ from the frozen Stage-1 network. Because each decision owns its subtree, credit is assigned per decision rather than per episode — the property that a sparse solved/unsolved reward lacks.

Policy and value.. On the frozen trunk of f_θ (variable embeddings $\mathbf{h}_j$) two heads are added. The policy logit is

$$g_j(s) = \log\left|\hat{p}_j - \tfrac{1}{2}\right| + h_\phi(\mathbf{h}_j), \qquad (4)$$

with h_ϕ a two-layer network whose last layer is initialized to zero, so that $\pi(j \mid s) \propto \exp(g_j/\tau)$ starts as a softening of (3) and its argmax is exactly rule (3). A value head $V_\psi(s)$ on a pooled graph embedding, $\log|K|$ and $\log m$ predicts $\log c$.

Update.. Each epoch samples training states, rolls the sampled policy ($\tau = 1$) out to a solution, and records every decision with its cost. With the advantage $\alpha(u) = V_\psi(s_u) - \log c(u)$ standardized within the epoch,

$$\mathcal{L}_2 = \sum_u \Big[-\alpha(u) \log \pi(j_u \mid s_u) - \lambda \mathcal{H}(\pi(\cdot \mid s_u)) + (V_\psi(s_u) - \log c(u))^2 \Big], \qquad \lambda = 0.01. \qquad (5)$$

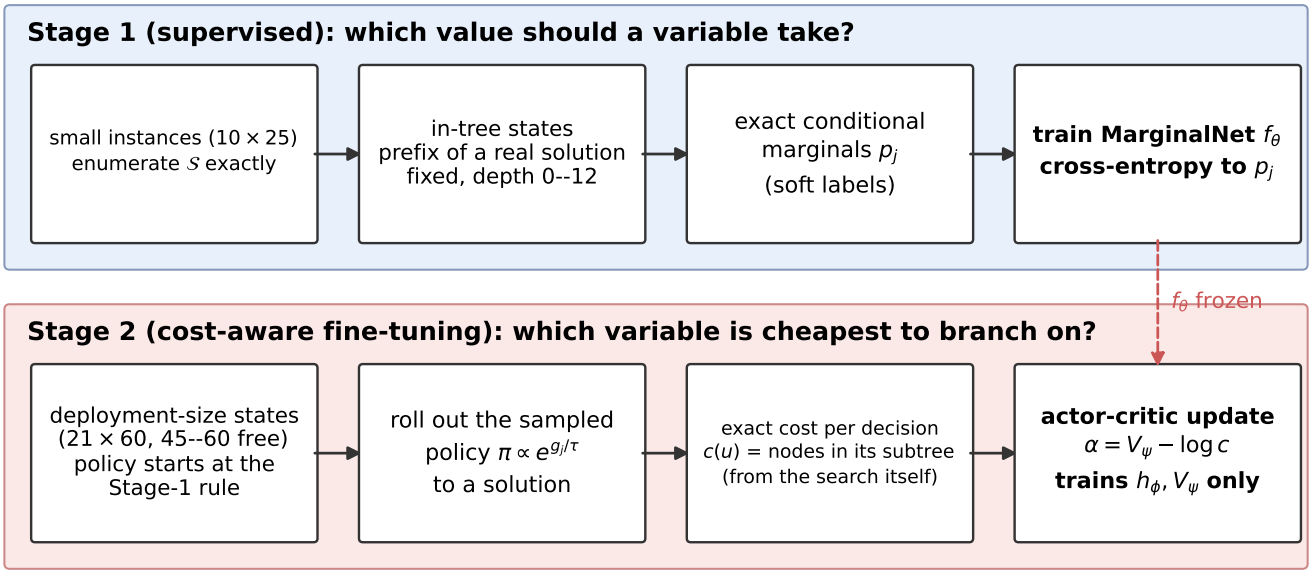


Figure 3: The two training stages. Stage 1 learns *which value* from exact conditional marginals of enumerable instances; Stage 2 learns *which variable* from the exact subtree cost of every decision, produced by the search, with the Stage-1 network frozen.

Only ϕ, ψ are trained; θ is fixed, so the marginal that chooses values and defines the FORCED/OPEN partition is unchanged, and any improvement is attributable to the branching order alone. The critic is pre-trained on decisions recorded under rule (3), and its held-out rank correlation with $\log c$ gates the actor update (a baseline that cannot rank costs makes the advantage noise). The epoch is selected on held-out states from the training pool, never on evaluation instances. At deployment the policy is greedy, $j^\star = \arg\max_j g_j(s)$ (Figure 1).

3.5. Cost per node

At a node with n_c free variables the network costs one relaxation plus one forward pass; LP-probing, the sound procedure that detects the same FORCED variables, costs n_c relaxations. With a warm-started relaxation on both sides the measured ratio grows from 1.7× at $n = 25$ to 7.2× at $n = 60$ and 17.8× at $n = 150$, but is only 1.3–1.7× at the depths where prediction matters on our instances, where probing additionally yields proofs. The Stage-2 head adds nothing measurable to the forward pass.

4. Experiments

4.1. Instances and protocol

Instances come from a market-split generator that plants a random $n/2$-of-n solution and, among 20 candidate plantings per A, keeps the one with the largest relaxation-vertex spread; no uniqueness is imposed. Every instance is feasible by construction: $\mathcal{S} \neq \emptyset$ is guaranteed by the planting, and deciding infeasibility is outside the scope of this paper (Section 5). Solution multiplicity changes the task — with $|\mathcal{S}| = 1$ every variable is FORCED — so m is chosen per size to match it: 10×25 (median $|\mathcal{S}| = 23$), 18×50 (19), 21×60 (10; 27/30 verified). At $n = 100$ a constraint solver finds no solution within 20 s, so search fails for every strategy and we stop at $n = 60$. Stage 1 trains on 10,000 in-tree states from 1,760 instances at 10 × 25 (Adam, 10^{-3}, 20 epochs, batch 1, gradient clipping 1.0; 20 minutes on one core); no hyperparameter was tuned and there is no validation split. Evaluation uses 30 instances per size, plus a 100-instance 21 × 60 set (generator seeds 700000–700099, containing the thirty) built after the 30-instance results were known. Stage 2 uses a separate 21 × 60 pool of 500 instances (seeds 800000+, 400/100 split), whose overlap with every evaluation set is zero by hash of $(A, \mathbf{b})$; its 100 held-out instances supply the states on which fine-tuning epochs are selected.

Every arm runs in the same loop: one warm-started relaxation per node (Gurobi dual simplex, bounds changed in place), identical propagation, identical value rule; arms differ only in the branching choice. Budgets are per-instance wall-clock, enforced by process termination; concurrency is 6 on a 20-core machine; all timing-sensitive code is single-threaded. Per-instance speed is tested with a two-sided sign test; node counts are seed-invariant because the search is deterministic given the relaxation vertex. Every 21 × 60 figure is replicated over three seeds. Enumeration and CP-SAT use OR-Tools; lattice reduction uses fpylll.

4.2. Prediction against the exact posterior

Table 1 places the Stage-1 network against the ceiling that no predictor can exceed, ceil $= \frac{1}{|K|}\sum_j \max(p_j, 1 - p_j)$. It closes most of the gap between relaxation rounding and the ceiling at every depth, and its 3–6 point margin over rounding does not vanish deep in the tree. The composition, not the level, is what moves: the FORCED share rises from 6.3% to 87.9%, so

	root	depth 8	all depths
conditional Bayes ceiling	70.8%	94.4%	87.2%
MarginalNet (in-tree trained)	66.7%	84.3%	**80.0%**
MarginalNet (root trained, controlled)	—	77.4%	75.4%
LP relaxation rounding	60.9%	80.9%	76.5%
FORCED share of variables	6.3%	87.9%	—

Table 1: Per-variable accuracy on held-out in-tree states of the 10×25 pool against the exact posterior. The controlled row trains on the same pool, labels, architecture and state count, on root states only.

almost every decision at depth 8 can cost a backtrack. Training on root states instead, with everything else controlled, gives a network that is *worse* than rounding inside the tree (75.4% against 76.4%): in-tree supervision is what makes learned guidance better than the classical rule. A graph-free MLP on the same features reaches 79.4% against the network's 84.1% on a matched split, so the graph structure carries part of the margin. Where the margin lives at deployment size is shown in Figure 4c: inside the training range (13–25 free variables) network and rounding are indistinguishable (correlation 1.000), because at constraint density $m/|K| \geq 0.8$ the relaxation is already integral on every FORCED variable; the entire margin (+5 points) arises at 50–60 free variables, outside the training range, where the relaxation is weakest ($m/|K| \approx 0.35$–0.42). 80% of the 21×60 search's queries fall there.

4.3. Search performance

Table 2 is the main result. Three points. First, Stage 1 produces a smaller tree than the LP rule at every size (2.0×, 3.9×, 5.5–9.5× fewer nodes) and the ratio grows with size, as Figure 4c predicts; a smaller tree buys time only once the tree is large enough to pay for the forward pass, so at 10×25 Stage 1 is slower. Second, the 100-instance set replicates the 21×60 result with power — Stage 1 is faster on 70–71/100 instances — at smaller margins than the original thirty (2.0–2.3× against 2.6–3.0×), which are the first thirty seeds of the larger set and a favourable sample for Stage 1; we take the 100-instance figures as the estimate. Third, Stage 2 removes a further 2.3–2.4× of the nodes and 2.0–2.4× of the time on 100 instances, in both training seeds and every evaluation seed, at unchanged per-node cost, for 4.8–5.9× over the LP rule end to end; the policy was fine-tuned at 21×60 only and transfers to 18×50 (1.7× fewer nodes, 1.8–2.2× faster). On the original thirty the two training seeds differ (one is near Stage 1, one 1.9× better), a spread the 100-instance set does not show. Random branching, for reference, solves 0/30 at 20×50 within 300 s and none of the 21×60 residual of Section 4.5.

4.4. Against complete solvers

The method does not compete with a complete solver (Table 3). The gap decomposes: at 21×60 CP-SAT's own branch counter reports a median of 18,827 branches, more than our 4,040–8,873 nodes, while it processes 19,782 nodes/s against our ~700 in Python, where the network's forward pass (1.9 ms at the root) is now over 90% of the per-node cost. The deficit is per-node throughput, not branching quality; whether it is recoverable with a compiled inference path and the clause learning and restarts our search lacks is not something we show.

4.5. Deployment in the full pipeline

Lattice coverage falls with size (30/30, 24–28/30, 10–16/30), which is what leaves room for branching to matter. At 21×60 the residual is genuinely hard — random branching closes none of the 50 residual instances pooled over seeds — and every sound strategy closes all of them; the difference is time. Stage 1 is 3.0× faster than the LP rule on the residual (40/50, $p = 2 \times 10^{-5}$) and Stage 2 a further 1.2–2.7× depending on the seed's residual. End-to-end solve rate is 100% for every sound arm and 44% for random branching.

4.6. Size transfer of both stages

Tables 5–7 evaluate every learned arm at sizes other than the one it was trained at; Table 5 restates what each arm is, Table 6 gives solve counts, times and nodes by size, and Table 7 reads the same numbers by transfer direction. Stage 1, trained only at 10×25, keeps a node advantage over the LP rule at every size (3.9×, 5.5×, 3.8×, 2.4×) and, with $|\mathcal{S}|$ matched, is not worse at 21×60 than a network trained at 18×50 (8,657 against 14,742 nodes, n.s.); its root accuracy holds its margin over rounding up to 40×100. But the time advantage of Stage 1 alone shrinks as the tree grows: at 24×70 it solves 29/30 against the LP rule's 23–25/30, yet on the instances both solve it is no faster (0.7–1.1×), because a 260,000-node tree pays for 260,000 forward passes. Stage 2 restores the advantage and carries it upward. The policy fine-tuned at 18×50 is 2.0× better than Stage 1 in nodes at 21×60 (69/100, $p < 0.001$; 1.7–1.9× in time); both fine-tuned policies solve 30/30 at 24×70 in every seed, 2.3–3.3× (22–23/30, $p \leq 0.016$) and 2.1–2.7× (19–21/30) faster than Stage 1 with 2.7× and 2.2× fewer nodes, at 1.2–1.4× their training size. At 28×80 the window closes for every arm we run: the LP rule and Stage 1 solve 4 and 8 of 30 within 1,200 s, the fine-tuned policies 18 and 17, and SCIP 30/30 in a median of 112 s. A policy fine-tuned on 45–60 free variables therefore doubles the solve rate at a size it never saw, but does not extend the method to where a complete solver is not needed. Two remarks. First, the fine-tuned policies gain nothing at their own training size when the tree is small (18×50: 1,147 against 1,085 nodes for the 18×50-trained policy), consistent with Figure 4c: what is learned matters on large, weakly constrained states, and those exist only in large trees. Second, at $n \geq 70$ SCIP overtakes CP-SAT, and both remain far ahead of us.

4.7. What the supervised target misses

Aligning the training distribution does not help.. Figure 4c suggests an obvious fix: train Stage 1 where its margin lives. We built 2,691 in-tree states with 30–60 free variables from the 21×60 training pool (prefix of the LP ordering fixed to the planted solution; exact marginals; all states with ≤ 55 free variables enumerate within 90 s, 291/400 roots do) and trained the same architecture on them alone (M1) and on a half-and-half

size (budget)	strategy	solve rate	median nodes	median time	nodes/s	vs. previous row (per instance)
10×25 (60 s)	LP	100%	46	0.03 s	1,767	
	STAGE 1	100%	24	0.07 s	376	slower: forward pass exceeds a 24-node tree
18×50 (300 s)	LP	100%	4,224	2.43 ± 0.33 s	2,066	
	STAGE 1	100%	1,085	1.82 ± 0.37 s	731	1.3–1.4×, 18/30, n.s.
	STAGE 1+2	100%	**424**	**0.52 ± 0.01 s**	820	1.8–2.2×, 22–24/30, $p \le 0.016$
21×60, 30 inst. (600 s)	LP	100%	82,366	47.9 ± 0.8 s	1,627	
	STAGE 1	100%	8,657	14.5 ± 0.1 s	619	2.6–3.0×, 23/30, $p = 0.005$
	STAGE 1+2 (seed 0 / 1)	100%	8,873 / **4,040**	12.5 / **5.9** s	710 / 690	1.2–1.5× n.s. / 2.3×, 20–21/30
21×60, 100 inst. (600 s)	LP	100%	79,609	41.2 ± 1.9 s	1,966	
	STAGE 1	100%	13,386	16.5 ± 0.6 s	796	2.0–2.3×, 70–71/100, $p < 0.001$
	STAGE 1+2 (seed 0 / 1)	100%	5,495 / **4,940**	7.6 / **6.8** s	738 / 730	2.0–2.2× / 1.9–2.4×, 64–68/100, $p \le 0.007$

Table 2: Branching strategies in the same warm-started loop, lattice stage off, three seeds (mean ± s.d. of per-seed medians; node counts are seed-invariant). LP branches on the most integral relaxation value; STAGE 1 is rule (3); STAGE 1+2 the cost-fine-tuned policy, trained at 21×60 only and reported for two training seeds. The last column compares each row with the row above it.

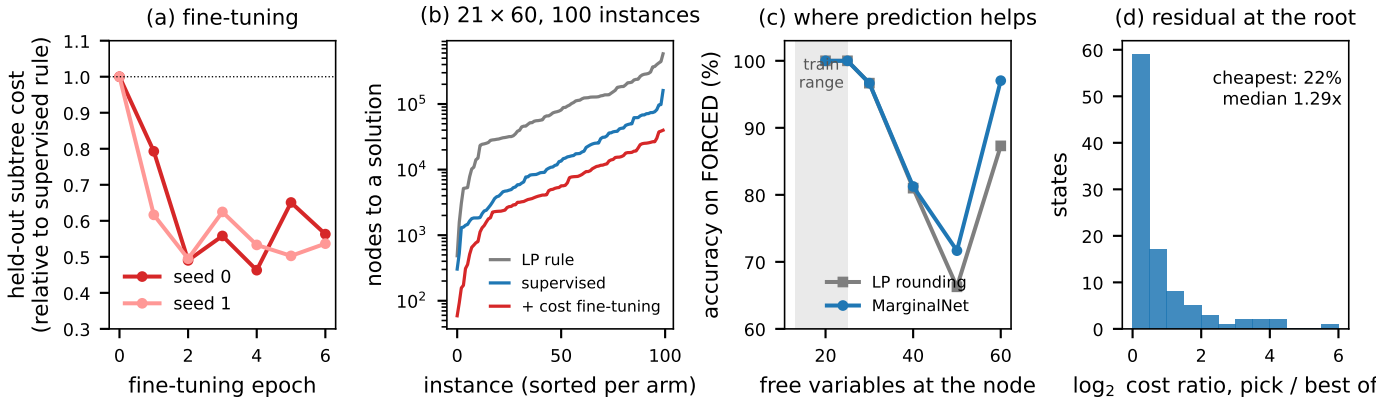


Figure 4: (a) Stage-2 fine-tuning: total subtree cost on 150 held-out states relative to the Stage-1 rule, two training seeds. (b) Nodes to a solution on the 100-instance 21×60 set, each arm sorted. (c) Accuracy on FORCED variables of 21×60 in-tree states by number of free variables: the Stage-1 margin over relaxation rounding lies outside the training range. (d) Residual at the root: cost of the confidence rule's choice relative to the cheapest of its five candidates (100 held-out states, 45–50 free variables).

size	CP-SAT	SCIP	stage 1	stage 1+2
10×25	**0.002 s**	0.009 s	0.07 s	—
18×50	**0.046 s**	0.385 s	1.82 s	0.52 s
21×60 (30)	**1.16 s**	3.69 s	14.5 s	5.9–12.5 s
21×60 (100)	**0.79 s**	2.59 s	16.5 s	6.8–7.6 s
24×70	16.1 s	**11.7 s**	358–457 s	125–185 s
28×80	270 s (23/30)	**112 s**	>1200 s (8/30)	720 s (18/30)

Table 3: Median solve time, single-threaded, same instances and budgets; solve counts are given where not all instances are solved. CP-SAT is 9–11× faster than the full method at 21×60 and 11× at 18×50; from 24×70 SCIP is the fastest, 10–15× ahead of the full method.

size	strategy	lattice	residual	time on residual
10×25	any	30/30	—	—
18×50	LP / STAGE 1 / 1+2	24–28/30	11/11	0.5–5.0 / 0.6–2.5 / 0.8–0.8 s
21×60	random	10–16/30	**0/50**	> 600 s
21×60	LP	10–16/30	50/50	52.9–65.1 s
21×60	STAGE 1	10–16/30	50/50	14.2–20.2 s
21×60	STAGE 1+2	10–16/30	50/50	**7.3–11.7 s**

Table 4: Cascade with lattice reduction first, three seeds; lattice coverage varies with the seed's column permutations, and every arm within a seed receives the identical residual. Median time on the residual instances, range over seeds.

mix (M2). Accuracy on FORCED variables at 50–60 free variables rises by about one point (+6.1/ + 10.7 against +5.4/ + 9.7 for Stage 1). The tree doubles: M1 expands 21,337 median nodes on the 100-instance set against 13,386 (fewer on 38/100, $p = 0.021$), M2 18,558, and a switch that routes nodes with ≥ 50 free variables to M1 recovers nothing (17,252). The reason is where the nodes are: two thirds of all decisions occur at 25–34 free variables, and on such small dense states M1's most confident variable is FORCED-and-correct 77.6% of the time against 94.4% for Stage 1 (rank correlation with the true margin 0.12 against 0.36). A network slightly better at the top of the tree and much worse inside it makes the search slower. Within 10×25, training on depths 0–6, 6–12 or 0–12 makes no difference (79.3/79.9/79.9%). The training-state distribution is not the lever.

The confident choice is rarely the cheapest.. On 100 held-out states with 45–50 free variables we forced the root to each of

arm	branches on	first value
LP	most integral relaxation value, $\arg\max_j \lvert x_j^{LP} - \frac{1}{2}\rvert$	rounded x_j^{LP}
STAGE 1 (25)	most confident marginal, $\arg\max_j \lvert\hat{p}_j - \frac{1}{2}\rvert$	$\mathbb{1}[\hat{p}_j \geq \frac{1}{2}]$
STAGE 1+2 (60 / 50)	cost-tuned score, $\arg\max_j g_j$	$\mathbb{1}[\hat{p}_j \geq \frac{1}{2}]$

Table 5: The branching arms. All run in the same depth-first loop (one warm-started relaxation per node, same propagation, same budget) and differ only in this rule; the number in parentheses is the size the learned component was trained at. LP is not a solver: it is the same loop with the relaxation value in place of the network.

the five most confident variables and ran rule (3) below. The rule's own choice is the cheapest of the five on 22% of states; the median cost ratio to the best candidate is 1.29, the mean 2.97, a candidate at least twice as cheap exists on 24% of states, and the best root choice alone would remove 55% of the nodes summed over these states (Figure 4d). The residual is about *which* variable, and it is large.

4.8. Cost-aware fine-tuning

Critic gate.. Recording every decision of rule (3) on the training pool gives 37,275 decisions (9,375 held-out) with exact subtree costs. The value head on the frozen trunk reaches a held-out Spearman correlation of 0.80 with $\log c$; a baseline that knows only the state's size reaches 0.70, and within size buckets the critic's advantage is larger (0.68 against 0.38 at 35–44 free variables). Unfreezing the trunk adds 0.02; the frozen critic is used.

Fine-tuning.. Six epochs of 400 rollouts from the 45–60 free-variable training states ($\tau = 1$, about 21,000 decisions per epoch). Figure 4a shows the held-out monitor: total subtree cost falls to 0.46 (seed 0, epoch 4 selected) and 0.50 (seed 1, epoch 2) of the Stage-1 rule's, and policy entropy from 1.7 to below 0.1. The evaluation is in Table 2 and Figure 4b.

What changed.. On 300 held-out states the fine-tuned policy branches on a different root variable than rule (3) at 95% of them; its pick is typically the 15th most confident ($\lvert\hat{p} - \frac{1}{2}\rvert$ of 0.26 against 0.43) and is FORCED with a correct first value less often (57.5% against 82.5%). The policy moved away from the variables the network is surest about, and the search got cheaper. The gain is not a property of the root decision alone: forcing only the root to the fine-tuned pick with rule (3) below gives a median cost ratio of 0.93 (cheaper on 22/46), so it is the sequence of decisions that improves. Two training seeds reach similar costs while choosing different variables at many nodes, as expected if the objective, not a particular solution, was learned. We can state what the fine-tuning optimizes but not yet name the feature it exploits; a cheap proxy, the propagation the first child triggers, does not differ between the two choices.

5. Limitations

Exact supervision bounds the method.. Marginals require enumerating $\mathcal{S}$, which does not complete beyond $n \approx 60$; Stage 1 transfers to larger sizes but cannot be trained there, and Stage 2's cost signal, while available at any size the search runs at, was used at 21×60 only.

The useful size window is bounded on both sides.. Below it lattice reduction closes everything; above it search fails: at 28×80 the fine-tuned policies solve 18/30 within 1,200 s and Stage 1 alone 8/30, and at $n = 100$ no strategy finds a solution. Our demonstration lives at $n = 50$–70. At $n \geq 70$ solution multiplicity cannot be verified by enumeration, so those sizes are matched on constraint density only, and 28×80 is a single seed.

The system is not competitive with CP-SAT. (9–11× slower), the deficit being per-node throughput in a Python loop dominated by the forward pass. Whether a compiled path plus clause learning closes it is not shown.

Statistical power and sampling.. The original thirty 21×60 instances proved a favourable sample for Stage 1 and an unstable one for Stage 2; the 100-instance set is the estimate we stand behind, and the cross-size results rest on thirty instances per size.

Stage 2 rests on two training pools and two seeds.. Two training seeds agree at 21×60 and the gain transfers to 18×50, 24×70 and, as a solve-rate gain, 28×80, but no policy was trained above $n = 60$. Its entropy collapses within four epochs, the epoch is selected by a monitor rather than a stopping rule, and the feature it exploits is unnamed. No hyperparameter of either stage was tuned; the depth range of Stage 1 was subsequently shown not to matter within a size.

Every instance is feasible; deciding infeasibility is out of scope.. The conditional marginal is undefined on an empty solution set, so Stage 1's supervision requires $\mathcal{S} \neq \emptyset$, and every instance we train or evaluate on has a planted solution. Finding a solution quickly and refuting an infeasible instance quickly are different objectives, and the second is where clause learning dominates. Stage 2's signal, by contrast, is defined either way — the cost of a decision is the size of its subtree whether that subtree contains a solution or is exhausted — so extending the method to infeasible instances is naturally a Stage-2 question, and it is untested.

Single instance family.. Whether either signal transfers to other constraint families is untested.

What is and is not new.. Exact marginals as a supervision target (NSNet) and subtree size as a per-decision return (FMSTS, tree MDPs) both exist; the method's novelty is in the conditional, guidance-only use of the first inside a complete search and in fine-tuning only the branching order on the second from a posterior-derived, frozen starting point. The empirical claim — that the two signals capture different things — is the paper's, and it rests on one instance family.

6. Conclusion

A branching decision asks two questions, and we trained on an exact answer to each. The exact conditional posterior — computable where solutions can be enumerated, and transported to every node by the reduction identity — tells the search which value to try and identifies which predictions can cost

	CP-SAT	SCIP	LP	STAGE 1 (25)	STAGE 1+2 (60)	STAGE 1+2 (50)
(a) instances solved within budget						
18×50 (300 s)	30/30	30/30	30/30	30/30	30/30	30/30
21×60, 100 inst. (600 s)	100/100	100/100	100/100	100/100	100/100	100/100
24×70 (1200 s)	30/30	30/30	23–25/30	29/30	**30/30**	**30/30**
28×80 (1200 s)	23/30	30/30	4/30	8/30	**18/30**	17/30
(b) median wall-clock time						
18×50	0.05 s	0.39 s	2.4 s	1.8 s	**0.5 s**	1.5 s
21×60, 100 inst.	0.8 s	2.6 s	41 s	16.5 s	**7.6 s**	10.7 s
24×70	16 s	12 s	537–644 s	358–457 s	**125–185 s**	135–165 s
28×80	270 s	112 s	>1200 s	>1200 s	720 s	852 s
(c) median nodes to a solution (seed-invariant)						
18×50	—	—	4,224	1,085	**424**	1,147
21×60, 100 inst.	—	—	79,609	13,386	**5,495**	7,796
24×70	—	—	980,743	260,114	**90,054**	95,818
28×80	—	—	1,929,041	803,535	**470,606**	560,077

Table 6: Size transfer of both stages, lattice off, 30 instances per size (100 at 21×60). 18×50, 21×60 and 24×70 are three seeds (ranges over seeds); 28×80 is one seed. Solution multiplicity is matched at $n \le 60$ and unverifiable at $n \ge 70$, where m is chosen for the constraint density of 21×60 ($m/n \approx 0.35$). Training sizes in parentheses (Table 5).

trained → tested	vs.	nodes	time	wins
STAGE 1: 25 → 50	LP	3.9×	1.3–1.4×	18/30
STAGE 1: 25 → 60	LP	5.5×	2.0–2.3×	70–71/100
STAGE 1: 25 → 70	LP	3.8×	0.7–1.1×	16–17/30
STAGE 2: 60 → 50 (down)	STAGE 1	2.6×	1.8–2.2×	22–24/30
STAGE 2: 50 → 60 (up)	STAGE 1	2.0×	1.7–1.9×	63–66/100
STAGE 2: 60 → 70 (up)	STAGE 1	2.7×	2.3–3.3×	22–23/30
STAGE 2: 50 → 70 (up)	STAGE 1	2.2×	2.1–2.7×	19–21/30
STAGE 2: 50 → 50 (same)	STAGE 1	0.9×	1.1–1.4×	16–18/30

Table 7: The same results read by transfer direction: ratio of median nodes and of per-instance median time (range over seeds), and the number of instances on which the transferred arm is faster. At 24×70 the supervised rule keeps its node advantage over LP but not its time advantage; the fine-tuned policies keep both.

anything; the exact subtree cost — produced by the search as it backtracks — tells it which variable to branch on. The first signal beats LP-guided branching by 2.0–2.3× on 100 hard 21×60 instances; the second, applied as a fine-tuning that leaves the first untouched, adds 2.0–2.4×, replicates across seeds, and transfers in size in both directions — at 24×70 it is what keeps the learned search ahead of the LP rule at all. That the fine-tuned policy prefers variables the posterior is less sure about, and that sharpening the posterior where its margin lives makes the tree larger, are the same fact seen twice: the probability of being right and the cost of being wrong are different objectives, and only the second is what a search minimizes.

Reproducibility

Code is organized under `proposed_src/` with a `ROLES.txt` per directory. Stage 1: `v15_bayes/v15_gen_by_m.py`, `v22_transfer/v22_split.py`, `v17_conditional/v17_make_conditional_data.py`, `v17_train_conditional.py`. Search and evaluation: `v23_ablation/run_revision2.sh`. Stage 2 and the residual experiments: `v24_deploy_range/run_v24.sh`, `run_v24_r1.sh`, `run_v24_r2.sh`, `run_v24_r2b.sh`; figures: `make_260917_figures.py`. All generators are seeded; instance data and weights are regenerated rather than distributed.

References

[1] Aardal, K., Hurkens, C. A. J., and Lenstra, A. K. (2000). Solving a system of linear Diophantine equations with lower and upper bounds on the variables. *Mathematics of Operations Research*, 25(3):427–442.

[2] Aardal, K., Bixby, R. E., Hurkens, C. A. J., Lenstra, A. K., and Smeltink, J. W. (2000). Market split and basis reduction: Towards a solution of the Cornuéjols–Dawande instances. *INFORMS Journal on Computing*, 12(3):192–202.

[3] Amos, B. (2023). Tutorial on amortized optimization. *Foundations and Trends in Machine Learning*, 16(5):592–732.

[4] Chen, Z., Liu, J., Wang, X., Lu, J., and Yin, W. (2023). On representing mixed-integer linear programs by graph neural networks. In *International Conference on Learning Representations*.

[5] Cornuéjols, G. and Dawande, M. (1999). A class of hard small 0-1 programs. In *Integer Programming and Combinatorial Optimization*, pages 284–293.

[6] Etheve, M., Alès, Z., Bissuel, C., Juan, O., and Kedad-Sidhoum, S. (2020). Reinforcement learning for variable

selection in a branch and bound algorithm. In *CPAIOR*, pages 176–185.

[7] Gasse, M., Chételat, D., Ferroni, N., Charlin, L., and Lodi, A. (2019). Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 32.

[8] Gershman, S. J. and Goodman, N. D. (2014). Amortized inference in probabilistic reasoning. In *Proceedings of the Annual Meeting of the Cognitive Science Society*, volume 36.

[9] Han, Q., Yang, L., Chen, Q., Zhou, X., Zhang, D., Wang, A., Sun, R., and Luo, X. (2023). A GNN-guided predict-and-search framework for mixed-integer linear programming. In *International Conference on Learning Representations*.

[10] Haralick, R. M. and Elliott, G. L. (1980). Increasing tree search efficiency for constraint satisfaction problems. *Artificial Intelligence*, 14(3):263–313.

[11] Khalil, E. B., Morris, C., and Lodi, A. (2022). MIP-GNN: A data-driven framework for guiding combinatorial solvers. In *AAAI Conference on Artificial Intelligence*.

[12] Khalil, E. B., Le Bodic, P., Song, L., Nemhauser, G., and Dilkina, B. (2016). Learning to branch in mixed integer programming. In *AAAI Conference on Artificial Intelligence*.

[13] Lenstra, A. K., Lenstra, H. W., and Lovász, L. (1982). Factoring polynomials with rational coefficients. *Mathematische Annalen*, 261(4):515–534.

[14] Li, Z. and Si, X. (2022). NSNet: A general neural probabilistic framework for satisfiability problems. In *Advances in Neural Information Processing Systems*, volume 35.

[15] Li, Z., Guo, J., and Si, X. (2023). G4SATBench: Benchmarking and advancing SAT solving with graph neural networks. *Transactions on Machine Learning Research*.

[16] Marques-Silva, J. P. and Sakallah, K. A. (1999). GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521.

[17] Montanari, A., Ricci-Tersenghi, F., and Semerjian, G. (2007). Solving constraint satisfaction problems through belief propagation-guided decimation. In *Allerton Conference on Communication, Control, and Computing*.

[18] Nair, V., Bartunov, S., Gimeno, F., et al. (2020). Solving mixed integer programs using neural networks. *arXiv:2012.13349*.

[19] Ohrimenko, O., Stuckey, P. J., and Codish, M. (2009). Propagation via lazy clause generation. *Constraints*, 14(3):357–391.

[20] Parsonson, C. W. F., Laterre, A., and Barrett, T. D. (2023). Reinforcement learning for branch-and-bound optimisation using retrospective trajectories. In *AAAI Conference on Artificial Intelligence*.

[21] Pesant, G., Quimper, C.-G., and Zanarini, A. (2012). Counting-based search: Branching heuristics for constraint satisfaction problems. *Journal of Artificial Intelligence Research*, 43:173–210.

[22] Refalo, P. (2004). Impact-based search strategies for constraint programming. In *Principles and Practice of Constraint Programming (CP)*, pages 557–571.

[23] Scavuzzo, L., Chen, F. Y., Chételat, D., Gasse, M., Lodi, A., Yorke-Smith, N., and Aardal, K. (2022). Learning to branch with tree MDPs. In *Advances in Neural Information Processing Systems*, volume 35.

[24] Cappart, Q., Moisan, T., Rousseau, L.-M., Prémont-Schwarz, I., and Cire, A. A. (2021). Combining reinforcement learning and constraint programming for combinatorial optimization. In *AAAI Conference on Artificial Intelligence*.

[25] Chu, G. and Stuckey, P. J. (2015). Learning value heuristics for constraint programming. In *CPAIOR*.

[26] Vaezipoor, P., Lederman, G., Wu, Y., Maddison, C., Grosse, R. B., Seshia, S. A., and Bacchus, F. (2021). Learning branching heuristics for propositional model counting. In *AAAI Conference on Artificial Intelligence*.

[27] Wassermann, A. (2025). Solving the market split problem with lattice enumeration. *arXiv:2508.08702*; to appear in *Mathematical Programming Computation*.

[28] Feng, S. and Yang, Y. (2025). SORREL: Suboptimal-demonstration-guided reinforcement learning for learning to branch. In *AAAI Conference on Artificial Intelligence*.

[29] Schnorr, C. P. and Euchner, M. (1994). Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical Programming*, 66:181–199.

[30] Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., and Dill, D. L. (2019). Learning a SAT solver from single-bit supervision. In *International Conference on Learning Representations*.

[31] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. (2018). Graph attention networks. In *International Conference on Learning Representations*.

[32] Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256.